

JAVA SPEZIAL

Streams, Lambdas und Filter

Funktionale Datenverarbeitung in Java

**Streams verstehen, Predicates nutzen, Lambdas sicher anwenden
Filter-Pipelines bauen, klassische Schleifen vergleichen**

In diesem Skript:

- Streams vs. Collections – Grundidee und Motivation
- Lambdas und funktionale Interfaces (z.B. `Predicate`)
- Einfache Filter mit Zahlen (gerade/ungerade usw.)
- Komplexe String-Filter mit kombinierten Predicates
- Filter mit eigenen Datentypen (z.B. Person-Objekte)
- Vergleich: Streams vs. `for-each` + `if`

Inhaltsverzeichnis

1	Einführung und Motivation	2
2	Streams – Grundidee	2
2.1	Collection vs. Stream	2
3	Wiederholung: Lambdas und funktionale Interfaces	2
3.1	Lambda-Syntax	2
4	Einfache Filter mit Zahlen	3
4.1	Gerade Zahlen mit Streams	3
4.2	Vergleich: for-each + if	4
5	Filter mit Strings	4
5.1	Was passiert hier?	5
6	Filter mit eigener Klasse Person	5
6.1	Die Klasse Person	5
6.2	Personen mit Streams filtern	6
6.3	Analyse des Beispiels	7
6.4	Stadt-Filter mit or	7
6.5	Vergleich: Streams vs. for-each + if	8
7	Streams vs. klassische Schleifen – Vor- und Nachteile	9
7.1	Vorteile von Streams	9
7.2	Nachteile / Herausforderungen von Streams	9
7.3	Wann was benutzen?	9
8	Fazit: Streams, Lambdas und Filter im Überblick	11
8.1	Was Sie jetzt können sollten	11
8.2	Typische Denkweise bei Streams	11
8.3	Prüfungsrelevante Muster	11
8.4	Ausblick	11
A	Prüfungsscheckliste: Streams, Lambdas und Filter	12
A.1	Begriffe und Konzepte	12
A.2	Streams anwenden	12
A.3	Lambdas und Predicates	12
A.4	Typische Aufgabentypen	12
A.5	Letzter Check vor der Prüfung	12
B	Mini-Formelsammlung: Streams, Lambdas, Predicates	13
B.1	Streams – typische Muster	13
B.2	Häufige Operationen	13
B.3	Lambdas – Kurzformen	13
B.4	Predicate – typische Filtermuster	14
B.5	Predicate kombinieren	14
B.6	Standard-Pipeline-Muster	14

1 Einführung und Motivation

Merkkasten

Streams und Lambdas sollen Ihnen helfen, Sammlungen von Daten (z.B. Listen) **kürzer, lesbarer und weniger fehleranfällig** zu verarbeiten als mit klassischen for-Schleifen.

Typische Situationen:

- Aus einer Liste nur bestimmte Elemente herausfiltern
- Daten umformen (z.B. String → Länge)
- Ergebnisse in einer neuen Liste sammeln

2 Streams – Grundidee

Ein **Stream** ist eine Folge von Elementen, auf die hintereinander Operationen wie `filter`, `map` und `collect` angewendet werden. Die ursprüngliche Collection (z.B. `List`) bleibt dabei unverändert.

2.1 Collection vs. Stream

Collection	Stream
Speichert Elemente (Datenbehälter)	Beschreibt eine Folge von Verarbeitungsschritten
Kann verändert werden (add/remove)	Ist einmalig, wird verbraucht
Wird mit Schleifen durchlaufen	Wird mit Operationen wie <code>filter</code> bearbeitet

3 Wiederholung: Lambdas und funktionale Interfaces

3.1 Lambda-Syntax

Listing 1: Beispiele für Lambdas

```
1 // Ein Parameter, Ausdruck
2 x -> x * 2;
3
4 // Typ explizit
5 (int x) -> x * x;
6
7 // Mit Block und return
8 (int x, int y) -> {
9     int sum = x + y;
10    return sum;
11 };
```

Merkkasten

Ein Lambda ersetzt immer genau ein **funktionales Interface** (z.B. `Predicate<T>` mit der Methode `boolean test(T t)`).

4 Einfache Filter mit Zahlen

4.1 Gerade Zahlen mit Streams

Listing 2: Gerade Zahlen mit Streams filtern

```
1 import java.util.Collection;
2 import java.util.List;
3 import java.util.function.Predicate;
4 import java.util.stream.Collectors;
5
6 public class EvenNumbersExample {
7
8     private static final int[] NUMBERS = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
9
10    public static void main(String[] args) {
11        // Array in eine Collection (z.B. List) umwandeln
12        List<Integer> baseCollection =
13            java.util.Arrays.stream(NUMBERS)
14                .boxed()
15                .collect(Collectors.toList());
16
17        // Methode aufrufen
18        Collection<Integer> evenNumbers =
19            findEvenNumbers(baseCollection);
20        System.out.println(evenNumbers);
21    }
22
23    public static Collection<Integer> findEvenNumbers(
24        Collection<Integer> baseCollection) {
25
26        Predicate<Integer> streamsPredicate =
27            item -> item % 2 == 0;
28
29        return baseCollection.stream()
30            .filter(streamsPredicate)
31            .collect(Collectors.toList());
32    }
33 }
```

Stolperfallen

- Ein Stream kann nur **einmal** durchlaufen werden.
- Nach einem Terminal-Operator wie `collect` ist der Stream verbraucht.

4.2 Vergleich: for-each + if

Listing 3: Die gleiche Logik ohne Streams

```
1 public static Collection<Integer> findEvenNumbersClassic(  
2     Collection<Integer> baseCollection) {  
3  
4     Collection<Integer> result = new java.util.ArrayList<>();  
5  
6     for (Integer item : baseCollection) {  
7         if (item % 2 == 0) {  
8             result.add(item);  
9         }  
10    }  
11    return result;  
12 }
```

Merkkasten

Mit Streams trennen Sie besser **Was** soll passieren (Bedingung) von dem **Wie** (Schleifen, Indizes).

5 Filter mit Strings

In diesem Beispiel filtern wir eine Liste von Wörtern mit Hilfe mehrerer `Predicate<String>`, die wir miteinander kombinieren.

Listing 4: Strings mit kombiniertem Predicate filtern

```
1 import java.util.Arrays;  
2 import java.util.List;  
3 import java.util.function.Predicate;  
4 import java.util.stream.Collectors;  
5  
6 public class PredicateStringExample {  
7  
8     public static void main(String[] args) {  
9         String[] WORDS = {  
10            "Baum", // B..., zu kurz, endet nicht mit n -> raus  
11            "Bahn", // zu kurz  
12            "Besen", // B...n, 5 Buchstaben -> rein  
13            "Balkon", // B...n, 6 Buchstaben -> rein  
14            "Bienen", // B...n, 6 Buchstaben -> rein  
15            "Boot", // zu kurz, endet nicht mit n  
16            "Bogen", // B...n, 5 Buchstaben -> rein  
17            "Haus" // faengt nicht mit B an, zu kurz  
18        };  
19  
20        // Array -> List  
21        List<String> baseCollection = Arrays.asList(WORDS);  
22  
23        // Predicates definieren  
24        Predicate<String> startsWithB = s -> s.startsWith("B");  
25        Predicate<String> endsWithN = s -> s.endsWith("n");  
26        Predicate<String> lengthAtLeast5 = s -> s.length() >= 5;  
27  
28        // Kombiniertes Predicate: B am Anfang, n am Ende, Laenge >= 5  
29        Predicate<String> complexFilter =  
30            startsWithB  
31            .and(endsWithN)
```

```

32 .and(lengthAtLeast5);
33
34 // Filtern und Ergebnis sammeln
35 List<String> filtered = baseCollection.stream()
36 .filter(complexFilter)
37 .collect(Collectors.toList());
38
39 System.out.println(filtered);
40 }
41 }

```

5.1 Was passiert hier?

- **Datenquelle:** Das Array `WORDS` wird mit `Arrays.asList` in eine `List<String>` umgewandelt.
- **Einzelne Predicates:**
 - `startsWithB`: Wort beginnt mit 'B'.
 - `endsWithN`: Wort endet mit 'n'.
 - `lengthAtLeast5`: Wortlänge mindestens 5 Zeichen.
- **Kombination:** Mit `and` werden die Bedingungen verknüpft: Nur wenn **alle** drei Predicates `true` zurückgeben, wird das Wort behalten.
- **Stream-Pipeline:** `baseCollection.stream().filter(complexFilter).collect(...)` filtert zuerst und sammelt dann alle passenden Wörter in einer neuen Liste.

Merkkasten

Mit `and`, `or` und `negate` lassen sich Predicates flexibel kombinieren:

- `p1.and(p2)` – beide Bedingungen müssen gelten
- `p1.or(p2)` – mindestens eine Bedingung muss gelten
- `p1.negate()` – Bedingung wird umgekehrt

So können Sie komplexe Filter **declarativ** formulieren, statt viele verschachtelte if-Abfragen zu schreiben.

6 Filter mit eigener Klasse Person

In vielen Programmen arbeiten wir nicht nur mit einfachen Typen wie `int` oder `String`, sondern mit eigenen Klassen (z.B. `Person`, `Kunde`, `Produkt`). Streams und Predicates funktionieren dort ganz genauso.

6.1 Die Klasse Person

Listing 5: Einfache Person-Klasse

```

1 public class Person {
2
3     private final String name;
4     private final int age;
5     private final String city;
6
7     public Person(String name, int age, String city) {
8         this.name = name;

```

```

9  this.age = age;
10 this.city = city;
11 }
12
13 public String getName() {
14     return name;
15 }
16
17 public int getAge() {
18     return age;
19 }
20
21 public String getCity() {
22     return city;
23 }
24
25 @Override
26 public String toString() {
27     return name + " (" + age + ", " + city + ")";
28 }
29 }

```

6.2 Personen mit Streams filtern

Wir legen eine kleine Liste von Personen an und filtern dann mit verschiedenen `Predicate<Person>`.

Listing 6: Personen nach Alter und Stadt filtern

```

1  import java.util.Arrays;
2  import java.util.List;
3  import java.util.function.Predicate;
4  import java.util.stream.Collectors;
5
6  public class PersonFilterExample {
7
8      public static void main(String[] args) {
9          List<Person> persons = Arrays.asList(
10             new Person("Anna", 17, "Berlin"),
11             new Person("Bernd", 22, "Berlin"),
12             new Person("Carla", 30, "Hamburg"),
13             new Person("David", 19, "München"),
14             new Person("Emma", 25, "Berlin")
15         );
16
17         // Predicates definieren
18         Predicate<Person> isAdult = p -> p.getAge() >= 18;
19         Predicate<Person> livesInBerlin =
20             p -> "Berlin".equals(p.getCity());
21
22         // Nur volljährige Personen
23         List<Person> adults = persons.stream()
24             .filter(isAdult)
25             .collect(Collectors.toList());
26
27         System.out.println("Volljährige: " + adults);
28
29         // Volljährige in Berlin
30         Predicate<Person> adultInBerlin =
31             isAdult.and(livesInBerlin);
32

```

```

33 List<Person> adultsInBerlin = persons.stream()
34 .filter(adultInBerlin)
35 .collect(Collectors.toList());
36
37 System.out.println("Volljährige in Berlin: " + adultsInBerlin);
38 }
39 }

```

6.3 Analyse des Beispiels

- **Datenbasis:** Eine `List<Person>` mit Name, Alter und Stadt.
- **Predicate<Person>:**
 - `isAdult`: Alter ≥ 18
 - `livesInBerlin`: Stadt ist genau "Berlin"
- **Kombination:** Mit `isAdult.and(livesInBerlin)` entsteht ein Predicate für "volljährig UND in Berlin".
- **Stream-Nutzung:** In beiden Fällen wird mit `stream()` gefiltert und das Ergebnis mit `collect(Collectors.toList())` wieder in eine Liste umgewandelt.

Merkkasten

Streams und Predicates sind nicht auf einfache Typen beschränkt. Sie können jede beliebige Klasse (z.B. `Person`, `Kunde`, `Produkt`) verwenden – Hauptsache, Ihre Bedingungen greifen auf die **Getter** oder Felder dieser Klasse zu.

6.4 Stadt-Filter mit or

Oft sollen mehrere Städte gleichzeitig erlaubt sein, zum Beispiel "Berlin ODER Hamburg". Das lässt sich mit `or` elegant formulieren.

Listing 7: Volljährige in Berlin ODER Hamburg

```

1 import java.util.function.Predicate;
2 import java.util.List;
3 import java.util.stream.Collectors;
4
5 //... Liste persons wie im vorherigen Beispiel...
6
7 // Predicates aus dem vorherigen Beispiel
8 Predicate<Person> isAdult = p -> p.getAge() >= 18;
9 Predicate<Person> livesInBerlin =
10 p -> "Berlin".equals(p.getCity());
11 Predicate<Person> livesInHamburg =
12 p -> "Hamburg".equals(p.getCity());
13
14 // Berlin ODER Hamburg
15 Predicate<Person> livesInBerlinOrHamburg =
16 livesInBerlin.or(livesInHamburg);
17
18 // Volljährig UND (Berlin ODER Hamburg)
19 Predicate<Person> adultInBerlinOrHamburg =
20 isAdult.and(livesInBerlinOrHamburg);
21
22 List<Person> adultsInBerlinOrHamburg = persons.stream()
23 .filter(adultInBerlinOrHamburg)
24 .collect(Collectors.toList());

```



```

25
26 System.out.println(
27     "Volljährige in Berlin oder Hamburg: "
28     + adultsInBerlinOrHamburg
29 );

```

Merkkasten

Mit `or` können Sie Bedingungen verbinden, bei denen **mindestens eine** Bedingung gelten soll:

- `livesInBerlin.or(livesInHamburg)`: Person lebt in Berlin **oder** in Hamburg.
- In Kombination mit `and` können Sie komplexe Bedingungen formulieren, z.B. `isAdult.and(livesInBerlin.or(livesInHamburg))` für "volljährig UND (Berlin ODER Hamburg)".

6.5 Vergleich: Streams vs. for-each + if

Die gleiche Filterlogik wie im Stream-Beispiel lässt sich auch klassisch mit einer for-each-Schleife und if-Bedingungen formulieren.

Listing 8: Volljährige in Berlin mit for-each und if

```

1 import java.util.ArrayList;
2 import java.util.Collection;
3 import java.util.List;
4
5 public class PersonFilterClassic {
6
7     public static void main(String[] args) {
8         List<Person> persons = List.of(
9             new Person("Anna", 17, "Berlin"),
10            new Person("Bernd", 22, "Berlin"),
11            new Person("Carla", 30, "Hamburg"),
12            new Person("David", 19, "Muenchen"),
13            new Person("Emma", 25, "Berlin")
14        );
15
16        Collection<Person> adultsInBerlin =
17            findAdultsInBerlin(persons);
18
19        System.out.println("Volljährige in Berlin: " + adultsInBerlin);
20    }
21
22    public static Collection<Person> findAdultsInBerlin(
23        Collection<Person> persons) {
24
25        Collection<Person> result = new ArrayList<>();
26
27        for (Person p : persons) {
28            if (p.getAge() >= 18 && "Berlin".equals(p.getCity())) {
29                result.add(p);
30            }
31        }
32
33        return result;
34    }
35 }

```

Merkkasten

Vergleich:

- **Klassische Variante:** Bedingung steckt direkt in der if-Abfrage. Die Logik ist im Schleifen-Code "eingebacken".
- **Stream-Variante:** Die Bedingung steckt in einem `Predicate<Person>` (z.B. `adultInBerlin`). Die Schleifen-Logik übernimmt der Stream.
- **Vorteil der Streams:** Filterbedingungen können leicht wiederverwendet und mit `and` / `or` kombiniert werden.

7 Streams vs. klassische Schleifen – Vor- und Nachteile

7.1 Vorteile von Streams

- **Kürzerer Code:** Viele Standardoperationen (Filtern, Abbilden, Sammeln) sind in einzelnen Pipelines ausdrückbar.
- **Bessere Lesbarkeit:** Der Code beschreibt eher „was soll passieren“ als „wie wird es im Detail durchlaufen“.
- **Wiederverwendbare Bedingungen:** Mit `Predicate<T>` usw. können Bedingungen in eigene Variablen/Funktionen ausgelagert und mehrfach verwendet werden.
- **Einfache Kombination von Bedingungen:** Durch `and`, `or` und `negate` lassen sich komplexe Filter klar und ohne verschachtelte `if`-Blöcke formulieren.
- **Unveränderlichkeit der Datenquelle:** Streams arbeiten typischerweise mit neuen Ergebnissammlungen, die ursprüngliche Collection wird nicht verändert.

7.2 Nachteile / Herausforderungen von Streams

- **Gewöhnungsbedürftige Denkweise:** Für Einsteiger kann die funktionale Schreibweise zunächst schwerer zu verstehen sein als eine einfache Schleife.
- **Debugging:** Das Schritt-für-Schritt-Debuggen in einer Stream-Pipeline ist oft unbequemer als in einer expliziten Schleife.
- **Übertreibung möglich:** Zu lange oder verschachtelte Pipelines können unübersichtlich werden – manchmal ist eine einfache Schleife klarer.
- **Einmalige Nutzung:** Ein Stream kann nur einmal verwendet werden. Nach einem Terminal-Operator (z.B. `collect`) ist er „verbraucht“.

7.3 Wann was benutzen?

- **Streams sind ideal, wenn**
 - Sie vor allem Daten **filtern, transformieren oder sammeln** wollen.
 - Sie klare, deklarative Filterbedingungen formulieren möchten.
 - Sie dieselben Bedingungen mehrfach oder in verschiedenen Kombinationen brauchen.
- **Klassische Schleifen sind oft sinnvoll, wenn**
 - viele **Seiteneffekte** vorkommen (z.B. Logging, Zwischenausgaben, komplexe Zustandsänderungen).

- der Ablauf sehr individuell ist und sich nicht gut als einfache Pipeline ausdrücken lässt.
- Sie Schritt für Schritt im Debugger durch den Code gehen und Werte verändern wollen.

8 Fazit: Streams, Lambdas und Filter im Überblick

In diesem Skript haben Sie die wichtigsten Bausteine für die funktionale Datenverarbeitung in Java kennengelernt: Streams, Lambdas und Predicates.

8.1 Was Sie jetzt können sollten

- Den Unterschied zwischen **Collections** und **Streams** erklären.
- Einfache Streams mit `filter`, `map` und `collect` schreiben.
- **Lambdas** als kompakte Implementierung funktionaler Interfaces (z.B. `Predicate<T>`) verwenden.
- Filterbedingungen für verschiedene Typen formulieren: `int`/`Integer`, `String` und eigene Klassen wie `Person`.
- Mehrere Predicates mit `and`, `or` und `negate` kombinieren.
- Streams mit klassischen `for-each`-Schleifen vergleichen und bewusst entscheiden, welche Variante Sie einsetzen.

8.2 Typische Denkweise bei Streams

Merkkasten

Statt direkt in Schleifen zu denken („Wie laufe ich durch “), denken Sie bei Streams eher in **Schritten**:

1. **Datenquelle** festlegen (z.B. `List<Person>`).
2. **Operationen** beschreiben (z.B. `filter` mit Predicates, `map` zur Umformung).
3. **Ergebnisform** wählen (z.B. `collect(Collectors.toList())`).

8.3 Prüfungsrelevante Muster

- **Gerade Zahlen filtern:** Stream über eine Liste von Integern, Predicate für `item % 2 == 0`.
- **String-Filter:** Kombination von Bedingungen wie „beginnt mit B“, „endet mit n“, „Mindestlänge“.
- **Eigene Klassen:** Filter auf Basis von Getter-Methoden (z.B. `Alter`, `Stadt`), kombiniert mit `and` / `or`.
- **Vergleich:** Eine Aufgabe mit Streams und die gleiche Aufgabe mit `for-each` + `if` formulieren können.

8.4 Ausblick

Wenn Sie mit diesen Grundlagen sicher umgehen, sind Sie gut vorbereitet für weiterführende Themen, zum Beispiel:

- weitere Stream-Operationen wie `map`, `flatMap`, `sorted`, `distinct`, `limit`, `skip`, `anyMatch`, `allMatch`.
- parallele Streams (`parallelStream()`).
- Kombination mit anderen funktionalen Interfaces (z.B. `Function`, `Consumer`, `Supplier`).

A Prüfungscheckliste: Streams, Lambdas und Filter

Diese Checkliste hilft Ihnen kurz vor der Prüfung zu prüfen, ob Sie die wichtigsten Punkte beherrschen.

A.1 Begriffe und Konzepte

- Kann ich den Unterschied zwischen **Collection** und **Stream** in eigenen Worten erklären?
- Weiß ich, was ein **funktionales Interface** ist (genau eine abstrakte Methode)?
- Kenne ich die Rolle von **Predicate<T>** und die Methode **boolean test(T t)**?
- Kann ich erklären, was ein **Terminal-Operator** ist (z.B. **collect**, **forEach**) und dass ein Stream danach verbraucht ist?

A.2 Streams anwenden

- Kann ich aus einer **List<Integer>** alle geraden Zahlen mit **stream() + filter + collect** herausfiltern?
- Kann ich das gleiche Problem mit einer **for-each**- Schleife und **if**-Bedingung lösen?
- Kann ich eine einfache Stream-Pipeline lesen und erklären, was Schritt für Schritt passiert?

A.3 Lambdas und Predicates

- Kann ich einfache Lambdas schreiben, z.B. **x -> x * 2** oder **p -> p.getAge() >= 18**?
- Kann ich aus einer verbalen Beschreibung (z.B. "volljährig und lebt in Berlin") ein **Predicate<Person>** formulieren?
- Weiß ich, wie man **and**, **or** und **negate** verwendet, um Bedingungen zu kombinieren?

A.4 Typische Aufgabentypen

- Zahlen filtern (z.B. gerade, größer als ein Schwellwert, zwischen zwei Grenzen).
- Strings filtern (z.B. beginnt mit Buchstabe X, Mindestlänge, bestimmtes Suffix).
- Eigene Klasse (z.B. **textttPerson**) nach mehreren Eigenschaften filtern (Alter, Stadt, Name).
- Einen gegebenen Schleifen-Code in eine Stream-Variante umschreiben (und umgekehrt).

A.5 Letzter Check vor der Prüfung

- Habe ich mindestens ein Beispiel mit **int/Integer**, eines mit **String** und eines mit einer eigenen Klasse (z.B. **Person**) selbst programmiert und getestet?
- Kann ich die wichtigsten Code-Beispiele aus diesem Skript grob aus dem Kopf skizzieren (ohne jede Klammer auswendig zu wissen)?
- Fühle ich mich sicher genug, eine unbekannte, aber ähnliche Aufgabe in der Prüfung logisch herzuleiten?

B Mini-Formelsammlung: Streams, Lambdas, Predicates

B.1 Streams – typische Muster

Grundmuster

```
1 // Aus Collection einen Stream machen
2 collection.stream();
3
4 // Filtern und in Liste sammeln
5 collection.stream()
6 .filter(predicate)
7 .collect(Collectors.toList());
8
9 // Elemente transformieren (map)
10 collection.stream()
11 .map(x -> /* neuer Wert */)
12 .collect(Collectors.toList());
13
14 // Auf jedes Element etwas ausführen (Side-Effect)
15 collection.stream()
16 .forEach(x -> System.out.println(x));
```

B.2 Häufige Operationen

Operation	Bedeutung
<code>filter(Predicate<T>)</code>	Elemente nach Bedingung auswählen
<code>map(Function<T,R>)</code>	Elemente in andere Werte umwandeln
<code>sorted()</code>	Elemente nach natürlicher Ordnung sortieren
<code>distinct()</code>	Doppelte entfernen
<code>limit(n)</code>	Nur die ersten n Elemente behalten
<code>skip(n)</code>	Die ersten n Elemente überspringen
<code>forEach(Consumer<T>)</code>	Aktion für jedes Element ausführen (Terminal)
<code>collect(Collectors.toList())</code>	Ergebnis in eine Liste sammeln (Terminal)
<code>count()</code>	Anzahl der Elemente zählen (Terminal)
<code>anyMatch(Predicate<T>)</code>	Mindestens ein Element erfüllt Bedingung?
<code>allMatch(Predicate<T>)</code>	Alle Elemente erfüllen Bedingung?

B.3 Lambdas – Kurzformen

Listing 9: Typische Lambda-Muster

```
1 // Ein Parameter, Ausdruck
2 x -> x * 2;
3
4 // Mit Typangabe
5 (int x) -> x * x;
6
7 // Zwei Parameter, Block
8 (int x, int y) -> {
9     int sum = x + y;
10    return sum;
11 };
12
13 // Zugriff auf Objekt-Methoden
14 s -> s.length();
```

```
15 p -> p.getAge() >= 18;
```

B.4 Predicate – typische Filtermuster

Predicate für verschiedene Typen

Zahlen (Integer)

```
1 Predicate<Integer> isEven = x -> x % 2 == 0;  
2 Predicate<Integer> greaterThan10 = x -> x > 10;
```

Strings

```
1 Predicate<String> startsWithB = s -> s.startsWith("B");  
2 Predicate<String> endsWithN = s -> s.endsWith("n");  
3 Predicate<String> minLength5 = s -> s.length() >= 5;
```

Eigene Klasse (Person)

```
1 Predicate<Person> isAdult = p -> p.getAge() >= 18;  
2 Predicate<Person> livesInBerlin =  
3   p -> "Berlin".equals(p.getCity());
```

B.5 Predicate kombinieren

Listing 10: Kombination von Predicates

```
1 // UND-Verknüpfung  
2 Predicate<Person> adultInBerlin =  
3   isAdult.and(livesInBerlin);  
4  
5 // ODER-Verknüpfung (Berlin oder Hamburg)  
6 Predicate<Person> livesInBerlinOrHamburg =  
7   livesInBerlin.or(livesInHamburg);  
8  
9 // Negation (nicht volljährig)  
10 Predicate<Person> isMinor = isAdult.negate();  
11  
12 // Beispiel-Stream  
13 List<Person> adultsInBerlin = persons.stream()  
14   .filter(adultInBerlin)  
15   .collect(Collectors.toList());
```

B.6 Standard-Pipeline-Muster

Merke: typische Prüfungs-Pipeline

```
1 List<Person> result = persons.stream()  
2   .filter(p -> p.getAge() >= 18) // filtern  
3   .filter(p -> p.getCity().equals("Berlin"))  
4   .map(Person::getName) // in Namen umwandeln  
5   .collect(Collectors.toList()); // in Liste sammeln
```

1. Datenquelle: `persons.stream()` 2. Zwischen-Operationen: `filter`, `map` 3. Terminal-Operation: `collect(...)`